

SIMATS ENGINEERING

Assessment Tool 1 — Analytical Problem Solving (CSA17: Artificial Intelligence)

Model Answers — Annexure A

Question 1 — The Water Jug Problem

Given: a 4-gallon jug and a 3-gallon jug, neither marked, and a pump for filling. Goal: get exactly 2 gallons into the 4-gallon jug.

Problem formulation

- State representation: (x, y) , where x = gallons currently in the 4-gallon jug (0–4) and y = gallons currently in the 3-gallon jug (0–3).
- Initial state: $(0, 0)$ — both jugs empty.
- Goal state: $(2, y)$ for any y — the 4-gallon jug contains exactly 2 gallons.
- Operators (production rules): Fill(4-gal), Fill(3-gal), Empty(4-gal), Empty(3-gal), Pour(4-gal $\rightarrow$ 3-gal) until 3-gal full or 4-gal empty, Pour(3-gal $\rightarrow$ 4-gal) until 4-gal full or 3-gal empty.
- Assumptions: no other measuring devices exist; water can be poured onto the ground; a jug can only be filled to its own capacity.

Solution — state-space search trace

Step	State (4-gal, 3-gal)	Explanation	Operator applied
0	(0,0)	Start — both jugs empty	-
1	(0,3)	Fill the 3-gallon jug	Fill(3-gal)
2	(3,0)	Pour 3-gallon jug into 4-gallon jug	Pour(3-gal $\rightarrow$ 4-gal)
3	(3,3)	Fill the 3-gallon jug again	Fill(3-gal)
4	(4,2)	Pour 3-gallon into 4-gallon until 4-gallon is full (only 1 gal fits, leaves 2 in 3-gal)	Pour(3-gal $\rightarrow$ 4-gal)
5	(0,2)	Empty the 4-gallon jug	Empty(4-gal)
6	(2,0)	Pour the remaining 2 gallons from 3-gallon jug into 4-gallon jug	Pour(3-gal $\rightarrow$ 4-gal)

Goal reached at Step 6: state $(2, 0)$ — the 4-gallon jug holds exactly 2 gallons, in 6 moves.

Question 2 — Mars Rover Agent

i. Percepts

- Camera images (panoramic/stereo images of terrain, rocks, sky)
- Spectrometer / chemical-sensor readings from soil and rock samples
- Distance/obstacle-sensor (LIDAR/ultrasonic) readings
- Wheel odometry and inertial navigation data (tilt, orientation)
- GPS-like positioning relative to landing site / orbiter triangulation
- Battery charge level and solar-panel power input
- Temperature and atmospheric sensor readings
- Commands/updates received from mission control on Earth

ii. Characterization of the operating environment (PEAS-style)

- Partially observable — the rover only senses its immediate surroundings via cameras/sensors, not the entire terrain.
- Stochastic — outcomes of actions (e.g., wheel slip on sand, dust storms) are not fully predictable.
- Sequential — earlier decisions (route taken, samples collected) affect what is possible later.
- Dynamic — weather, lighting, and terrain conditions (e.g., dust storms) change independently of the rover's actions.
- Continuous — sensor readings, position, and movement are continuous-valued, not discrete.
- Single-agent from the rover's perspective (it does not compete or coordinate with another acting agent in real time), though it communicates with mission control.
- Unknown (at least initially) — the detailed terrain map is not fully known in advance and is built up as the rover explores.

iii. Actions available to the agent

- Move forward/backward, turn left/right, adjust wheel speed
- Extend/retract robotic arm; drill or scoop a soil/rock sample
- Activate onboard spectrometer/chemical analysis instruments
- Capture and store images
- Transmit collected data to an orbiter or directly to Earth
- Avoid or navigate around detected obstacles
- Enter low-power/recharge mode using solar panels

iv. Performance evaluation

- Number and scientific value of samples successfully collected and analyzed
- Distance covered safely without collisions, tip-overs, or getting stuck
- Quality and completeness of data/images transmitted back to Earth
- Energy efficiency — power consumed relative to power harvested
- Overall mission longevity (operational days/years) and target-goal completion (e.g., reaching designated survey sites)

v. Most suitable agent architecture

A hybrid model-based, goal-based (with elements of utility-based) agent architecture is most suitable. The rover cannot rely on a simple reflex agent alone because the environment is only partially observable — it needs an internal model of the world (terrain map, own position, obstacle history) to make sensible decisions, which requires model-based reasoning. On top of that internal model, the rover must plan multi-step routes to reach scientifically interesting sites and return samples, which is a goal-based capability (formulating a goal and searching for a route to it). Because there are trade-offs between competing objectives — e.g., minimizing energy use vs. maximizing scientific value vs. minimizing risk of damage — a utility function layered on top (utility-based agent) allows the rover to rank actions rather than only accept/reject them, which matches how real Mars rovers (e.g., Curiosity, Perseverance) autonomously navigate: reflex-level obstacle avoidance combined with model-based mapping and goal/utility-driven mission planning.

Question 3 — The 8-Queens Problem

Goal: place 8 queens on an 8×8 chessboard so that no two queens attack each other (no shared row, column, or diagonal).

Problem formulation (incremental formulation)

- States: any arrangement of 0 to 8 queens on the board, at most one queen per column, with queens placed in columns left to right.
- Initial state: an empty board (no queens placed).
- Actions / successor function: add a queen to the left-most empty column, in any row of that column that is not attacked by a queen already placed.
- Goal test: 8 queens are placed on the board and no two queens attack each other.
- Path cost: irrelevant here — every step costs the same (1), so only the final configuration matters, not the path taken to reach it.

Why the example configuration is not a solution

Two queens attack each other along a diagonal whenever $|\text{row}_i - \text{row}_j| = |\text{col}_i - \text{col}_j|$ for their positions $(\text{row}_i, \text{col}_i)$ and $(\text{row}_j, \text{col}_j)$. In the given figure, the queen in the first column and the queen in the last column satisfy this equality, so they lie on the same diagonal and attack one another — hence the configuration violates the goal test even though it fills every column.

Search strategy

- Backtracking search (depth-first search with constraint checking): place queens column by column; whenever a placement conflicts with an existing queen (same row or diagonal), backtrack and try the next row in that column. This prunes the search space early instead of generating all 8^8 complete boards.
- Local search alternative (Min-Conflicts): start with all 8 queens placed (one per column, random rows), then repeatedly move the queen involved in the most conflicts to the row that minimizes conflicts, until no conflicts remain — this scales much better for larger N-Queens instances than systematic backtracking.

Question 4 — OLA Cab Booking Agent

Type of problem-solving agent

This is best modelled as a Goal-based / Utility-based problem-solving agent. The customer's goal is fixed (reach the destination), but there are multiple ways to satisfy it — different cab types (mini, micro, sedan, shared, prime) trading off cost, waiting time, and comfort. A simple goal-based agent would only check whether a plan reaches the goal; because the app must rank competing options against each other, a utility function (combining fare, ETA, capacity, and rating) is layered on top, making this a utility-based agent built on a goal-based search.

Pseudocode

```
function OLA-CAB-AGENT(percept) returns an action
  persistent: seq    // an action sequence, initially empty
                state // the agent's current understanding of the world
                goal  // the destination to be reached
                problem // the problem formulation

  state <- UPDATE-STATE(state, percept)           // current & drop location, live
traffic
  if seq is empty then
    goal    <- FORMULATE-GOAL(destination)
    cabTypes <- GET-AVAILABLE-CAB-TYPES(state)    // mini, micro, sedan, shared,
prime, ...
    for each cab in cabTypes do
      cab.fare <- ESTIMATE-FARE(cab, state, goal)
      cab.eta  <- ESTIMATE-ETA(cab, state)
      cab.utility <- UTILITY(cab.fare, cab.eta, cab.comfortRating, cab.capacity)
    bestCab <- ARGMAX(cabTypes, cab.utility)
    problem <- FORMULATE-PROBLEM(state, goal, bestCab) // route to destination
    seq      <- SEARCH(problem)                       // e.g., UCS/A* over the road
network
    action <- FIRST(seq)
    seq    <- REST(seq)
  return action
```

At each cycle the agent re-evaluates: if traffic conditions or cab availability change materially (a percept update), it re-runs the cab-selection and route-search steps rather than blindly continuing the old plan — this is the standard SIMPLE-PROBLEM-SOLVING-AGENT template adapted with a utility-based selection step for choosing among cab types.

Question 5 — Least-Cost Delivery Route (Uniform Cost Search)

Graph edges (directed, with cost) read from the network diagram:

From	To	Cost
S	A	1
S	G	12
A	B	3
A	C	1
B	D	3
C	D	1
C	G	2
D	G	3

UCS trace

UCS expands the lowest cumulative-cost node from the frontier (a priority queue ordered by $g(n)$); the goal test is applied only when a node is popped for expansion, not when it is merely generated.

Step	Node expanded	$g(n)$	Action / children generated	Frontier after step
1	S	0	Expand S $\rightarrow$ generate A (cost 1), G (cost 12)	{A:1, G:12}
2	A (cost 1)	1	Expand A $\rightarrow$ generate B ($1+3=4$), C ($1+1=2$)	{C:2, B:4, G:12}
3	C (cost 2)	2	Expand C $\rightarrow$ generate D ($2+1=3$), G via C ($2+2=4$) — replaces worse G:12	{D:3, G:4, B:4}
4	D (cost 3)	3	Expand D $\rightarrow$ G via D would be $3+3=6$, worse than existing G:4 $\rightarrow$ discarded	{G:4, B:4}
5	G (cost 4)	4	G popped from frontier — Goal test passed $\rightarrow$ STOP	GOAL REACHED

Least-cost path and cost

Optimal path: $S \rightarrow A \rightarrow C \rightarrow G$

Total cost = $1 + 1 + 2 = 4$

This beats the direct $S \rightarrow G$ edge (cost 12), the $S-A-C-D-G$ route (cost 6), and the $S-A-B-D-G$ route (cost 10), confirming $S-A-C-G$ as the guaranteed least-cost path that UCS finds.